Khiaty — Reverse-Engineering the System (W4D2)

A custom-tailoring marketplace: customers post a request, tailors bid, one bid is accepted and paid, the piece is made, and the order is closed out. Everything below is reasoned from using the live app as a customer (Dana) and a tailor (Mariam) — no source code.

Part 1 — What Is This App Remembering? (Data)

Every "thing" the app has to store for it to work at all:

1. **User / account** — email, password, and a **role** (customer vs tailor). The same login leads to two different apps depending on this one field.
2. **Tailor profile** — bio, rating (5.0★), review count, "pieces completed", and a **portfolio** of finished work. (A customer profile is lighter — mostly just a name.)
3. **Request** — title, piece type (Abaya, Thobe, …), description, budget range, "needed by" date, status (OPEN), and the customer who owns it.
4. **Reference image** — the photo uploaded with a request, stored as a file somewhere separate from the request row (it comes back as a URL).
5. **AI-suggested description** — the text the app drafts *from* the photo, stored alongside the request as its own field (separate from whatever the customer finally types).
6. **Bid** — which request, which tailor, an **upfront** amount, an **on-completion** amount, a turnaround in days, an optional note, and a status (PENDING / DECLINED / ACCEPTED).
7. **Bid revision** — *this is the non-obvious one.* Every bid keeps a full list of its past numbers with timestamps ("Revision 1, 2, 3 …"). This isn't a screen you go to; it has to exist as stored rows for the "see how this bid changed" history to render.
8. **Order** — created only once a bid is accepted *and paid*. Has a status that moves through stages, the two payments, the finished-piece photos, and the turnaround.
9. **Payment** — the upfront and final payments, each marked paid/unpaid. The order's very existence depends on a payment being confirmed.
10. **Message** — chat tied to an order (sender, text, time).
11. **Review** — rating + text a customer leaves on a completed order; it feeds the tailor's public rating and portfolio.

A relationship, in my own words (Request ↔ Bid): One **request** can have **many bids** (the ivory abaya had 3; my test request started at 0, then 1). One bid belongs to exactly one request and one tailor. The number is **not fixed** — it grows over time as tailors discover the request in their "Browse" feed and place bids, and each tailor can hold at most one (revisable) bid per request. So it's one-request-to-many-bids, and the "many" keeps changing until the request is closed.

Does a revision overwrite or add? I revised a bid and then looked at what the customer can see. The old numbers were **still there** — every revision is listed with its own date, and only the newest is labelled "CURRENT". Mariam's bid on the ivory abaya had **15** revisions visible to Dana. So a revision **adds a new record and keeps the old ones** (append-only history); it does not overwrite. The "current price" is really just "the latest row in a list the app never throws away."

Permanent record vs. current value:

- **Permanent / can't undo:** a **completed order and its review.** Once Layla's gown was finished, paid in full, and reviewed ("The fit was exact…"), that review is locked in and permanently feeds Mariam's rating and portfolio. There's no edit/undo on it.
- **Just the "current" value:** a **bid's price.** It changes every time the tailor revises — it's the mutable head of a list. Same with a request still sitting at status OPEN: it's a live, changeable thing until it turns into an order.

Part 2 — What's Actually Running Behind This? (Technology)

Reference-photo upload — where is it processed? I uploaded a photo. It was stored in about **1.4 seconds** and came back as a URL on a **Supabase storage** domain — i.e. it was **sent to a server/bucket**, not kept in the browser. Then I pressed "Suggest a description," which took about **9 seconds** and called a server endpoint (`/api/ai/analyze-image`). That long, one-off delay is the tell: a browser can't "look at" an image and write a paragraph about fabric and trim — the photo is shipped to a **server-side vision model** that reads it and returns text. If it were pure browser work it would be instant and wouldn't need a network round-trip.

Chat showing up without a refresh: The messages on a paid order arrive live. That needs a **persistent server-push connection (WebSocket)** — here, Supabase **Realtime** subscribing to database changes. A plain page reload can't do this: reload only shows what the server had *at the moment you asked*. To see the other person's message the instant they send it, the server has to be able to push to you without you asking — which is exactly what a WebSocket/Realtime channel gives and a request/ response reload cannot.

The moment you click to pay: "Accept & Pay" submits to a payments endpoint and the order page states it plainly: *"This order exists because the upfront payment was confirmed with MyFatoorah — not because anyone clicked accept."* So the browser hands off to **MyFatoorah** (an external Kuwaiti payment processor); card details go to *them*, not to Khiaty. A small team builds it this way so they **never touch raw card numbers** — that offloads PCI-DSS compliance and fraud liability to the processor.

One login, two experiences — what decides which? A stored **role** on the account. Signing in as a customer lands on "My requests"; signing in as a tailor lands on "Browse requests / My bids." This decision belongs **on the server**, because the browser can't be trusted to pick its own permissions — if the client decided, anyone could flip a flag and walk into the other role. The server reads the role tied to your session and routes (and gates data) accordingly.

Part 3 — Why Does It Behave This Way? (System Logic)

Post → bid → decline → try to revise the declined bid: I posted a request, bid on it as a tailor, and on a separate request that was already DECLINED I tried to **revise the declined bid**. The app **accepted the revision** — it saved new numbers and appended them to the history. So "declined" is **not the end of the road**: a declined bid isn't frozen; the tailor can still come back with new numbers (a soft, reversible state, not a locked terminal one). It behaves like "the customer said no *for now*," leaving the door open for the tailor to re-pitch.

Trying to see another tailor's exact bid: As a tailor I can see only **"bids placed: 3", "lowest total", and "fastest turnaround."** Never a name, never anyone's exact number. I checked whether the real numbers were merely hidden in the page and found the competitors' names and their individual amounts were **not sent to my browser at all** — only the aggregates. This is a deliberate **sealed/blind bidding** design: if tailors could see each other's quotes they'd just shave a fraction off the lowest, start a race to the bottom, or collude. Hiding the real numbers keeps each bid an honest, independent quote.

Reaching chat before a bid is accepted and paid: There is **no chat at all** until an **order** exists, and an order only exists after the upfront payment clears. The Orders page for an unpaid request is empty ("An order opens once a bid is accepted and its upfront payment clears"). So two people can only message once there's **money committed** — it stops tailors and customers from taking the deal off-platform before Khiaty is in the loop, and keeps the chat attached to a real transaction.

Stages of an order: `ACCEPTED → IN PROGRESS → READY → COMPLETED`. What moves it along is **not all one kind of thing**: the jump into an order in the first place is gated by a **payment confirmation** (the MyFatoorah event, not a button), and the final "paid in full / completed" is gated by the **final payment**. The middle steps (in progress → ready, uploading the finished photos) look like **the tailor clicking** to advance. So some transitions are a person's action and at least two are gated by money actually changing hands.

Part 4 — Where Would This Break? (Trust)

I picked actions involving **money, privacy, and access**, and ran only safe, read-only checks (no deleting, no touching other people's data).

Hypothesis A — "Blind bidding is probably just hidden in the UI." If the server actually sends every bid to my browser and only *hides* the rivals with CSS/JavaScript, then the real numbers are one DevTools inspection away. *What I tried:* as a tailor, I inspected the data the ivory-abaya page actually delivered to my browser, looking for the other tailors' names and their exact amounts. *Result:* they **weren't there** — only the aggregates (count, lowest total, fastest turnaround) were sent. The backend genuinely withholds the raw bids. **Access control passes this one.**

Hypothesis B — "Maybe a customer-only page is reachable by URL as a tailor." (IDOR) The customer's "Accept & Pay / Decline" screen lives at `/requests/<id>`. If the server only *hides the link* from tailors but still serves the page, I could open it by typing the URL. *What I tried:* while logged in as tailor Mariam, I navigated straight to the ivory abaya's customer management URL (a request I don't own). *Result:* the server **redirected me to** `/browse` and served no accept/decline controls. The page isn't just hidden — the **backend refuses it by role.** Passes.

Hypothesis C — "Can the client dictate the price it pays?" *What I observed:* the "Accept & Pay" form sends only the **bid's id and the payment type (`upfront`) — no amount.** So the server looks up the price itself from the database; there's no client-supplied number to tamper with. Combined with the fact that the order is created from the **payment processor's confirmation** (not a client click), forging "paid" or paying a self-chosen amount from the browser shouldn't work.

Takeaway: on the three things that matter — seeing rivals' bids, reaching a page you shouldn't, and setting your own price — the frontend is a **thin display over a backend that re-checks everything.** The UI hides options, but the server is what actually enforces them.

Your Systems Analysis

Data model, roughly: A `user` has a **role** that forks the whole experience. A `customer` owns many `requests`; each `request` has one optional `reference_image` (in object storage) and an AI-drafted description. A `request` collects many `bids`; each `bid` belongs to one `tailor` and owns an append-only list of `bid_revisions` (the full price history the customer can see). When a bid is accepted **and its upfront `payment` is confirmed**, one `order` is born, linked to that request/bid/customer/tailor. An `order` has a forward-only status (ACCEPTED→IN PROGRESS→READY→COMPLETED), two `payments`, a thread of `messages`, finished-piece photos, and ultimately one `review` — which flows back into the tailor's rating and portfolio.

Technologies doing the real work: Next.js (React with server-rendered pages and server actions — forms POST to the server and redirect) on **Vercel**; **Supabase** for the database, for **file storage** (reference images), and for **Realtime** (the live chat over WebSockets); a **server-side AI vision model** behind `/api/ai/analyze-image` for drafting descriptions from a photo; and **MyFatoorah** as the external payment processor, so Khiaty never handles card data.

The single most deliberate design decision: making **an order come into existence from a confirmed payment rather than from a click.** That one choice quietly enforces half the system's rules at once — it's why chat can't be reached early, why a tailor can't fake an accepted deal, and why "accept" and "money in" can never drift apart. Blind bidding is the cleverest *visible* decision, but payment-as-the-source-of-truth is the sharpest *structural* one.

Start Your Build — brief for my own version

A two-sided made-to-order marketplace:

- **Two roles** off one login, routed and gated on the server.
- **Customers** post a request (type, description, budget, deadline, optional reference photo with an AI-drafted description).
- **Tailors** browse open requests and place **blind bids** (upfront + on-completion + turnaround + note), **revisable with full append-only history** the customer can see; rivals' numbers are never sent to the client.
- **Accept & pay** through an external processor; the **order is created from the payment confirmation**, not the click.
- **Chat unlocks only after payment**, over a realtime channel.
- **Order lifecycle** ACCEPTED→IN PROGRESS→READY→COMPLETED, closed out with finished photos, final payment, and a **review** that builds the tailor's rating + portfolio.

For each new piece I build, I'll re-ask Part 3 and Part 4 against my own app: is this rule enforced on the server, or only hidden in my UI?

Appendix — My build and its OWASP security audit

Build: https://github.com/t068580-ai/Ghurza. Ghurza is my own version of the system analysed above (Next.js 16, Prisma + SQLite, server-side authorization everywhere, blind bidding enforced in SQL, orders created only from a signed payment callback). **Audit:** the OWASP security skill from github.com/agamm/claude-code-owasp (commit 1ae29cb), run on the project with its own five-step workflow and report format. **First run:** 0 critical, 0 high, 3 medium, 4 low. **After fixes:** two medium findings are fixed (the login-lockout bypass at window rollover and the decodable blind-bid ranges), one medium is partly fixed (login DoS: the rest needs per-client limits at a reverse proxy), and a missing COOP header was added. The auditor re-checked every fix, found one more edge case, and I fixed that too. All 25 automated security tests pass. **Still open:** 1 medium and 4 low, all listed in Figure 2.

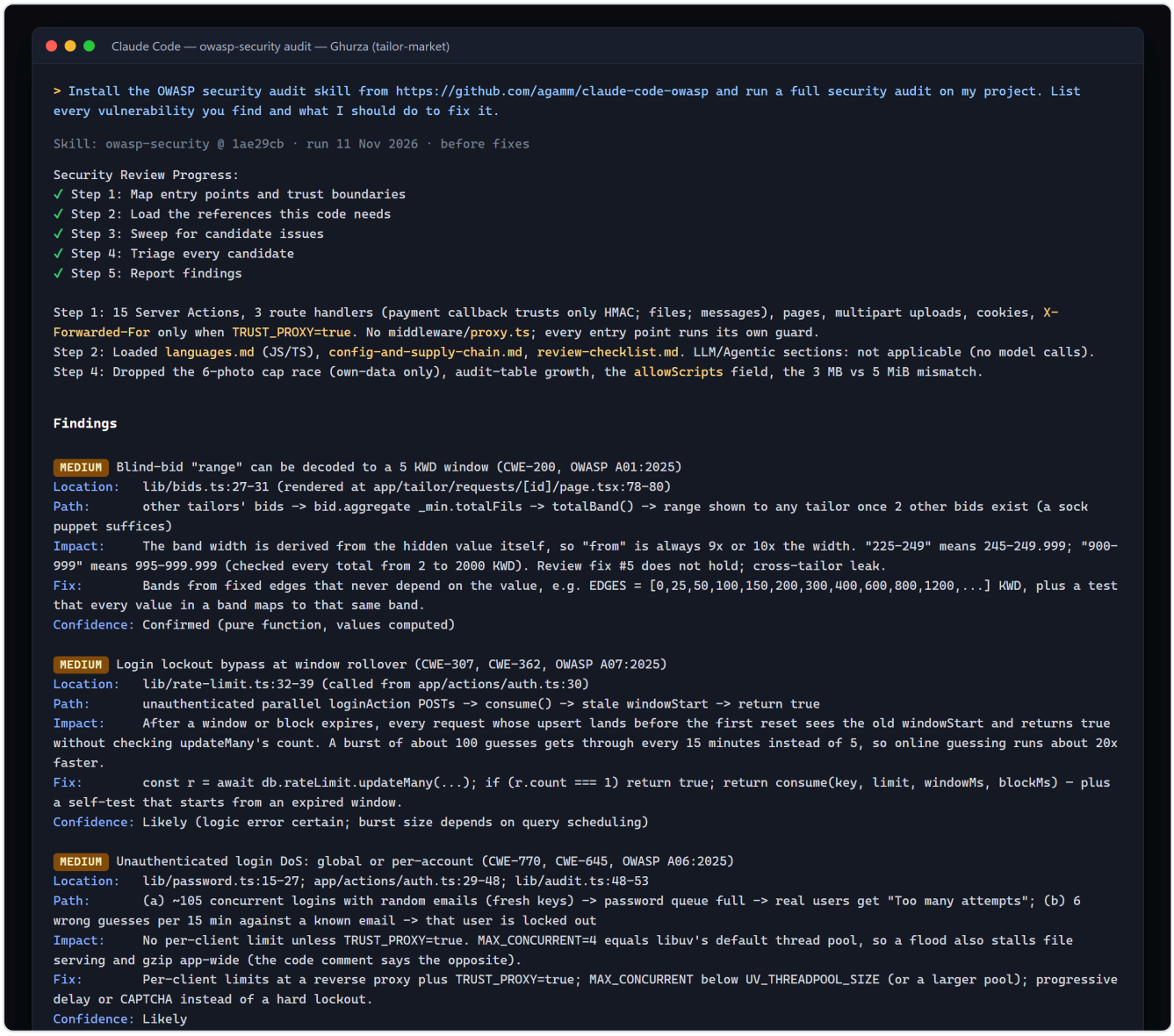


Figure 1a — Audit output before fixes (1 of 2): progress checklist, entry-point mapping and the three Medium findings.

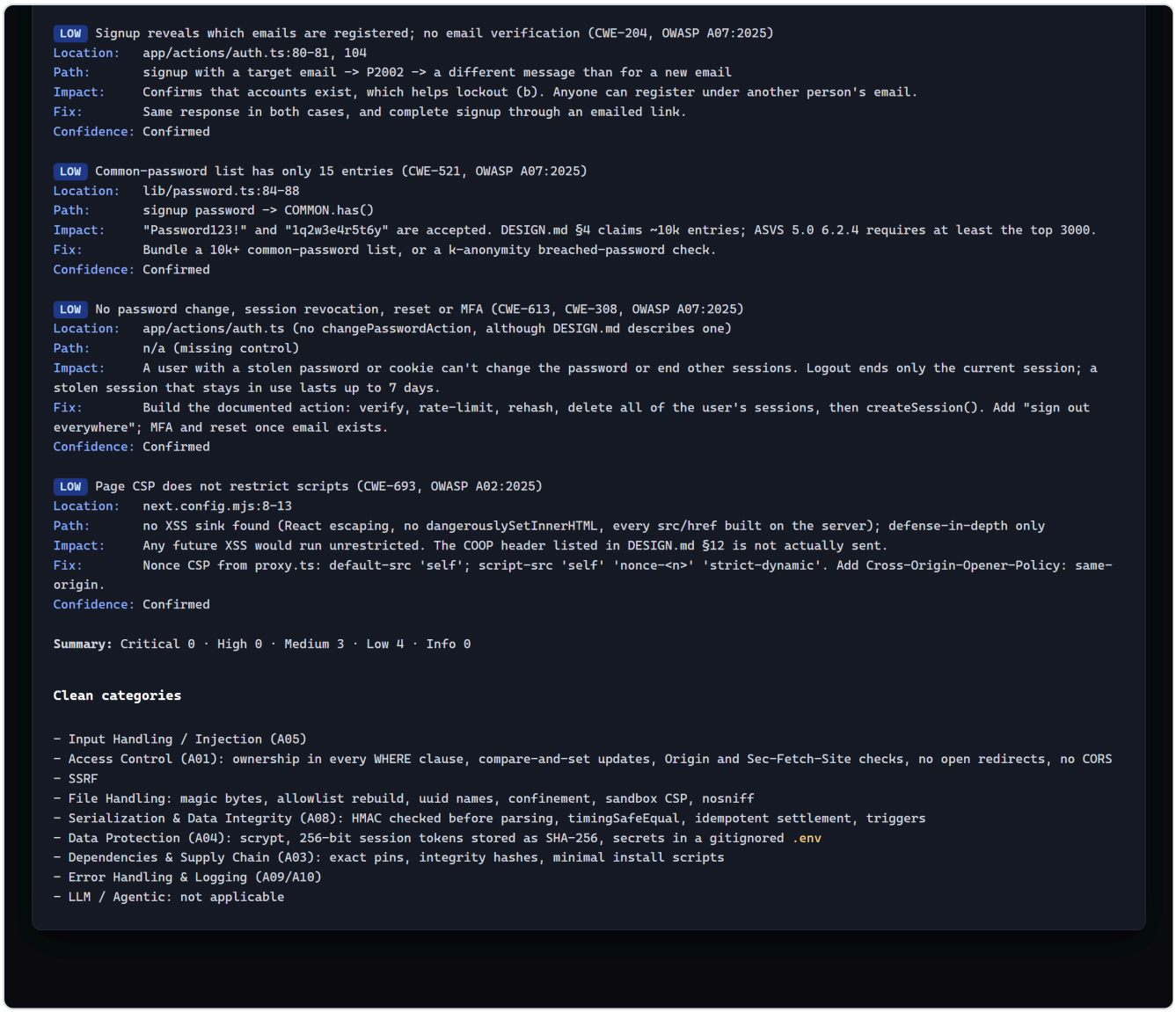


Figure 1b — Audit output before fixes (2 of 2): the four Low findings, summary count and clean categories.



Figure 2 — Re-check after fixes: status of every finding, the follow-up fix and the test results.